DSA Lab 5 (0093 sahasra marreboyina)

1.Insertion at the beginning:

Code-

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void insertAtStart(int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

int main() {
    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));

    n1->data = 10; n1->next = n2;
    n2->data = 20; n2->next = n3;
    n3->data = 30; n3->next = n4;
    n4->data = 40; n4->next = NULL;
    head = n1;

    printf("Before: %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data, head->next->next->next->data);

    insertAtStart(5);

    printf("After:  %d -> %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data,
            head->next->next->next->data, head->next->next->next->next->data);

    return 0;
}
```

Output:

```
Before: 10 -> 20 -> 30 -> 40 -> NULL
After:  5 -> 10 -> 20 -> 30 -> 40 -> NULL
```

2.Insertion at the end of link list:
Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void insertAtEnd(int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

int main() {
    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));

    n1->data = 10; n1->next = n2;
    n2->data = 20; n2->next = n3;
    n3->data = 30; n3->next = n4;
    n4->data = 40; n4->next = NULL;
    head = n1;

    printf("Before: %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data, head->next->next->next->data);

    insertAtEnd(50);

    printf("After:  %d -> %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data,
            head->next->next->next->data, head->next->next->next->next->data);

    return 0;
}
```

Output:

```
Before: 10 -> 20 -> 30 -> 40 -> NULL
After:  10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

3.Insertion at a given position
: Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void insertAtStart(int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void insertAtPosition(int value, int position) {
    if (position < 1) {
        printf("Invalid position\n");
        return;
    }
    if (position == 1) {
        insertAtStart(value);
        return;
    }
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    struct Node *temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of range\n");
        free(newNode);
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
```

```c
int main() {
    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));

    n1->data = 10; n1->next = n2;
    n2->data = 20; n2->next = n3;
    n3->data = 30; n3->next = n4;
    n4->data = 40; n4->next = NULL;
    head = n1;

    printf("Before: %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data, head->next->next->next->data);

    insertAtPosition(25, 3);

    printf("After:  %d -> %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data,
            head->next->next->next->data, head->next->next->next->next->data);

    return 0;
}
```

Output:

```
Before: 10 -> 20 -> 30 -> 40 -> NULL
After:  10 -> 20 -> 25 -> 30 -> 40 -> NULL
```

1. Deletion at Beginning:

Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void deleteAtStart() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    free(temp);
}

int main() {
    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));


    n1->data = 10; n1->next = n2;
    n2->data = 20; n2->next = n3;
    n3->data = 30; n3->next = n4;
    n4->data = 40; n4->next = NULL;

    head = n1;
    printf("Before: %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data, head->next->next->next->data);

    deleteAtStart();

    printf("After:  %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data);

    return 0;
}
```

Output:

```
Before: 10 -> 20 -> 30 -> 40 -> NULL
After:  20 -> 30 -> 40 -> NULL
```

2.Deletion at End:

Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }
    struct Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

int main() {

    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));

    n1->data = 10; n1->next = n2;
    n2->data = 20; n2->next = n3;
    n3->data = 30; n3->next = n4;
    n4->data = 40; n4->next = NULL;
    head = n1;

    printf("Before: %d -> %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data, head->next->next->next->data);

    deleteAtEnd();

    printf("After:  %d -> %d -> %d -> NULL\n",
            head->data, head->next->data, head->next->next->data);
```

Output:

```
Before: 10 -> 20 -> 30 -> 40 -> NULL
After:  10 -> 20 -> 30 -> NULL
```